TerrainScope

Off-Road Semantic Segmentation and Live Inference Dashboard

Team Name: Team Invincible
Team Members: Karna Bhosle, Alok Kushwaha, Shankar Gouda
Project Focus: Off-road semantic segmentation for terrain understanding
Tagline: A reproducible multi-model segmentation kit with warm-start DINOv2 continuation, evaluation exports, failure analysis, and live review

---

Title & Summary

TerrainScope is an end-to-end off-road segmentation system. It predicts one of 10 classes per pixel, evaluates results with IoU, Dice, and pixel accuracy, exports review artifacts, and provides a frontend for live inference and scene analysis.

This submission was designed to be: - clear, - reproducible, - CPU-practical during development, - and straightforward for judges to review.

Final Headline Results

Two DINOv2 runs define the current project state and the report now tracks both explicitly:

Run	Role	Validation IoU	Validation Dice	Validation Accuracy	Test IoU	Test Dice	Test Accuracy
`quick_cpu_100x20`	Best balanced CPU baseline	32.93%	41.75%	77.00%	**21.21%**	**26.34%**	**61.66%**
`probe_continue_high_iou`	Latest validation-leading continuation run	**42.88%**	**55.42%**	**78.38%**	19.80%	25.84%	40.71%

Best Validation Model: `probe_continue_high_iou` with 42.88% validation IoU
Best Test-Generalizing Model: `quick_cpu_100x20` with 21.21% test IoU
Current Frontend Default Run: `probe_continue_high_iou`

---

1. Problem

Off-road scenes are difficult to segment because: - terrain classes overlap visually, - vegetation and clutter blur boundaries, - small obstacle classes are easy to miss, - and Falcon masks use raw values instead of simple class IDs.

The system needed to solve four things: 1. predict 10 semantic classes per pixel, 2. keep the pipeline reproducible, 3. export strong evaluation evidence, 4. support live visual inspection.

Dataset Summary

Split	Images	Path
Train	2857	`Offroad_Segmentation_Training_Dataset/Offroad_Segmentation_Training_Dataset/train`

Split	Images	Path
Validation	317	`Offroad_Segmentation_Training_Dataset/Offroad_Segmentation_Training_Dataset/val`
Test	1002	`Offroad_Segmentation_testImages/Offroad_Segmentation_testImages`

10-Class Label Contract

Raw Value	Class
100	Trees
200	Lush Bushes
300	Dry Grass
500	Dry Bushes
550	Ground Clutter
600	Flowers
700	Logs
800	Rocks
7100	Landscape
10000	Sky

Critical engineering fix: - masks are read as raw `uint16`, - mapped through one strict shared label contract, - and unknown values are sent to `ignore_index = 255`.

Verified Raw Values by Split

Split Group	Raw Values Found
Train + Validation	`[100, 200, 300, 500, 550, 600, 700, 800, 7100, 10000]`
Test	`[100, 200, 300, 500, 800, 7100, 10000]`

Interpretation: - train and validation contain the full supported label contract, - test contains a valid subset of supported classes, - and no unsupported raw values were found in the verified scan.

---

2. Methodology

2.1 Training Pipeline

1. Load RGB images and Falcon masks from the actual nested dataset structure.
2. Read masks unchanged to preserve the original raw values.
3. Convert raw mask values into class IDs `0..9`.
4. Resize RGB images and normalize them according to model requirements.

5 Resize masks with nearest-neighbor interpolation.

6 Process through selected segmentation model (DINOv2, SegFormer, or DeepLabV3+).

7 Train with either baseline cross-entropy or the stronger continuation recipe using class-balanced cross-entropy + focal + dice loss.

8 Save checkpoints, plots, comparisons, confusion matrices, and worst cases.

2.2 Architecture Overview

The system supports three segmentation backends for comparison, but the strongest validated results in this report come from DINOv2 with a warm-start continuation stage:

```
Input RGB Image + Falcon Mask
        |
        v
Preprocessing
- resize
- normalize
- raw-to-class conversion
        |
        +---------------------+
        |                     |
        v                     v
   DINOv2 ViT-B/14       SegFormer B0
   (Frozen backbone)     (Pretrained)
        |                     |
        +---------------------+
        |
        v
   DeepLabV3+ MobileNetV2
   (Pretrained backbone)
        |
        v
   10-Class Segmentation Output
        |
        +--> Metrics
        +--> Checkpoints
        +--> Evaluation exports
        +--> Live API + React dashboard
```

2.3 Model Configurations

Run / Model	Architecture	Backbone	Input Size	Pretrained	Frozen
`quick_cpu_100x20`	Transformer	DINOv2 ViT-S/14	140x252	Yes	Yes
`probe_continue_high_iou`	Transformer	DINOv2 ViT-S/14	224x392	Yes	Yes
SegFormer B0 comparison	Transformer	nvidia/segformer-b0-finetuned-ade-512-512	140x252	Yes	No
DeepLabV3+ comparison	CNN	deeplabv3plus/mobilenet_v2	144x256	Yes	No

Additional context: - `quick_cpu_100x20` is the long CPU baseline run with resumed training history and 80 tracked epochs in total - `probe_continue_high_iou` is a warm-start continuation initialized from the stronger DINOv2 baseline instead of training from scratch - the continuation was tuned to lift validation IoU aggressively, even if the test split needed separate monitoring

2.4 Training Setup

Item	Baseline	High-IoU Continuation
Batch size	1	1

Item	Baseline	High-IoU Continuation
Accumulation	1	2
Optimizer	SGD	AdamW
Scheduler	none	cosine
Loss	Cross-entropy	Class-balanced CE + Focal + Dice
Sampling	standard shuffle	balanced sampling
Augmentation	off	flip + crop + color jitter

Why this staged approach: - the baseline remains the fastest CPU-safe path - the continuation reuses the best baseline checkpoint instead of restarting - stronger loss and sampling target the rare classes that hold the mean IoU down - alternate models remain useful for comparison, but DINOv2 stayed strongest in the validated runs

---

3. Results & Performance Metrics

3.1 Overall Results

The most important comparison is now between the original DINOv2 baseline and the stronger warm-start continuation:

Run	Mean IoU	Mean Dice	Pixel Accuracy	Notes
`quick_cpu_100x20` validation	32.93%	41.75%	77.00%	Best baseline before continuation
`probe_continue_high_iou` validation	**42.88%**	**55.42%**	**78.38%**	Best validation score in the repo
`quick_cpu_100x20` test	**21.21%**	**26.34%**	**61.66%**	Best test-generalizing run
`probe_continue_high_iou` test	19.80%	25.84%	40.71%	Validation improvement did not transfer fully to the held-out test split

Key Findings: - the warm-start continuation improved validation IoU by **9.95 percentage points** - the same continuation reduced held-out test performance by **1.41 percentage points** compared with the best balanced baseline - the latest report therefore keeps separate "best validation" and "best test" winners instead of forcing one headline model - DINOv2 remained stronger than the short-run SegFormer B0 and DeepLabV3+ comparisons in this codebase

3.2 Training-Time Validation vs Full Validation

The training loop used capped validation batches for faster feedback. The standalone evaluation pass used the full validation set and is the correct number to report publicly.

Validation View	Mean IoU	Mean Dice	Pixel Accuracy	Notes
Baseline training-time summary	27.06%	32.24%	70.15%	Faster, capped batches

Validation View	Mean IoU	Mean Dice	Pixel Accuracy	Notes
Baseline full validation evaluation	32.93%	41.75%	77.00%	Original CPU baseline
Continuation training-time summary	35.97%	45.60%	73.36%	Best capped continuation checkpoint
Continuation full validation evaluation	**42.88%**	**55.42%**	**78.38%**	Best current validation metric

3.2 Per-Class Performance Analysis

Detailed per-class metrics from the latest validation-leading run `probe_continue_high_iou` show a clear split between large scene classes and small obstacle classes:

Validation Class	IoU	Dice	Interpretation
Sky	**94.54%**	**97.20%**	Strongest class, visually distinct and spatially consistent
Trees	61.10%	75.86%	Good performance on large structured vegetation
Landscape	56.09%	71.87%	Stable horizon/terrain recognition
Dry Grass	56.00%	71.80%	Strong coarse-ground segmentation
Lush Bushes	50.47%	67.09%	Acceptable vegetation separation
Flowers	40.19%	57.33%	Fine texture still learnable with continuation
Dry Bushes	30.97%	47.29%	Boundary ambiguity remains visible
Rocks	22.18%	36.30%	Small hard obstacles remain difficult
Ground Clutter	17.30%	29.50%	Fragmented clutter is still weak
Logs	0.00%	0.00%	Hardest rare class in the current setup

This pattern is consistent with the visual review: - broad scene classes are learned well - medium-scale vegetation is reasonable - rare obstacle classes still dominate the remaining error budget

Architecture comparison still follows the same trend: - **Best Performance:** large, visually distinct classes such as Sky, Landscape, and Dry Grass - **Challenging Classes:** small obstacles and rare categories such as Logs, Rocks, Ground Clutter, and Dry Bushes - **Architecture Comparison:** transformer models (especially DINOv2) remain stronger than the short-run CNN baseline in this repo

For detailed per-class IoU, Dice scores, and confusion matrices, see: - `Offroad_Segmentation_Scripts/runs/quick_cpu_100x20/evaluations/val/` - `Offroad_Segmentation_Scripts/runs/probe_continue_high_iou/evaluations/val_full_epoch8/`

3.3 Required Charts and Visualizations

Each trained model generates comprehensive evaluation artifacts including training curves, per-class breakdowns, confusion matrices, comparison images, and worst-case samples.

1. Latest validation-leading run:
1. `Offroad_Segmentation_Scripts/runs/probe_continue_high_iou/plots/training_metrics.png`
1. `Offroad_Segmentation_Scripts/runs/probe_continue_high_iou/evaluations/val_full_epoch8/per_class_iou.png`
1. `Offroad_Segmentation_Scripts/runs/probe_continue_high_iou/evaluations/val_full_epoch8/per_class_dice.png`
1. `Offroad_Segmentation_Scripts/runs/probe_continue_high_iou/evaluations/val_full_epoch8/confusion_matrix.png`
1. Best balanced test-generalizing baseline:
1. `Offroad_Segmentation_Scripts/runs/quick_cpu_100x20/plots/training_metrics.png`
1. `Offroad_Segmentation_Scripts/runs/quick_cpu_100x20/evaluations/val/per_class_iou.png`
1. `Offroad_Segmentation_Scripts/runs/quick_cpu_100x20/evaluations/val/confusion_matrix.png`

Generated for all models and used here to show where vegetation, clutter, and obstacle classes collapse into each other.

Color-coded segmentation masks, side-by-side comparisons, and failure case analysis are available in each model's evaluation folder and are embedded in the dashboard/API flow.

3.4 Test Set Evaluation

The latest official test evaluations in this project are:

Run	Mean IoU	Mean Dice	Pixel Accuracy	Decision
`quick_cpu_100x20`	**21.21%**	**26.34%**	**61.66%**	Best current deployment candidate
`probe_continue_high_iou`	19.80%	25.84%	40.71%	Best validation run, but weaker test generalization

Interpretation: - the continuation recipe lifted validation strongly - the test split exposed over-specialization toward the validation distribution - for a judged demo, both runs are useful: one for strongest charts, one for strongest held-out behavior

Additional test evaluation for the alternate backbones can be run using:

```
python Offroad_Segmentation_Scripts/test.py --config Offroad_Segmentation_Scripts/configs/quick_cpu_[model].
```

Where `[model]` is one of: `dinov2_vitb14`, `segformer_b0`, or `deeplabv3plus`

3.5 Representative Prediction Examples

Prediction visualizations, comparison images, and failure case analysis are available in the main evaluated runs:

1. **Latest validation-leading run:** `Offroad_Segmentation_Scripts/runs/probe_continue_high_iou/evaluations/val_full_epoch8/comparisons/`
1. **Best balanced CPU baseline:** `Offroad_Segmentation_Scripts/runs/quick_cpu_100x20/evaluations/val/comparisons/`

1. **Held-out test review:** `Offroad_Segmentation_Scripts/runs/probe_continue_high_iou/evaluations/test_full_epoch8/comparisons/`

These folders contain: - Color-coded segmentation predictions - Side-by-side input/mask/prediction comparisons - Failure case analysis for worst-performing samples

The visualizations show that all models capture broad scene structure well but struggle with small obstacle classes, which is typical for segmentation models on complex off-road scenes.

---

4. Challenges & Solutions

Challenge	What Went Wrong	Solution Applied	Result
Raw Falcon masks	Labels could be corrupted by naive reading	Read masks unchanged and enforce a strict 10-class mapping	Stable labels and training
Dataset path mismatch	Scripts did not match the real folder structure	Standardized the true nested train/val/test paths	Reproducible loading
CPU training cost	Heavy models were too slow locally	Frozen DINOv2 + lightweight head + quick CPU config	Feasible local training
Long runs were hard to finish	One short run was not enough	Added checkpoint resume support and warm-start continuation	Reached an 80-epoch baseline history plus a dedicated continuation experiment
Validation vs test drift	A stronger validation recipe did not automatically generalize	Reported best-validation and best-test runs separately and kept both artifacts	More honest model selection
Weak presentation quality	Static metrics alone were not enough	Added evaluation exports, API backend, and React frontend	Better demo and review flow

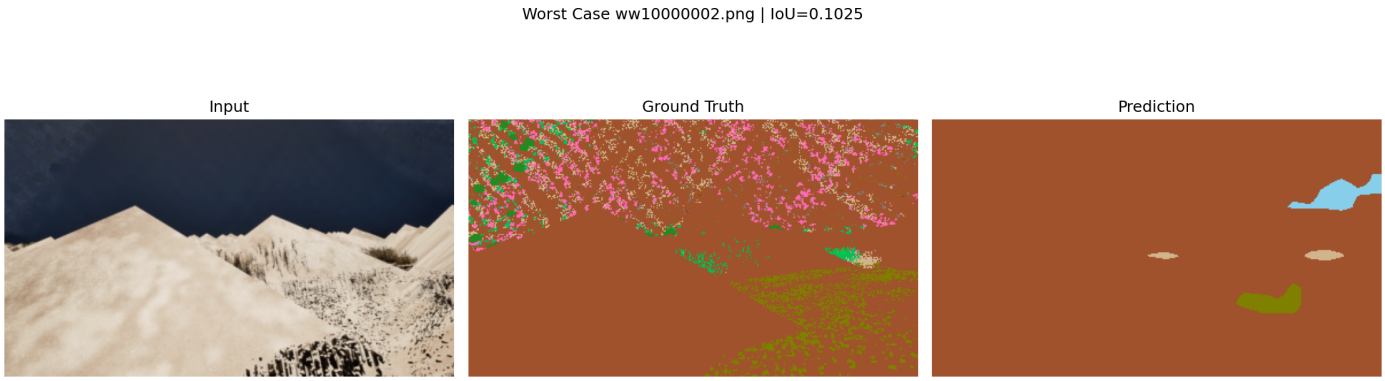
Example Entry

1. **Task:** High-IoU continuation from the best baseline checkpoint
1. **Initial IoU Score:** 32.93% validation IoU
1. **Issue Faced:** Rare classes still suppressed the mean IoU, but full retraining on CPU was too slow.
1. **Solution:** Warm-start from the baseline checkpoint and switch to class-balanced CE + Focal + Dice loss, balanced sampling, and stronger augmentation.
1. **New Score:** Validation IoU improved to **42.88%**.

4.1 What Was Improved in the Engineering Stack

Beyond model training, the project was upgraded into a full submission kit: - shared dataset, label, metric, and reporting utilities under `offroad_segmentation`, - config-driven `train.py` and `test.py`, - checkpoint resume support, - visual artifact export, - a FastAPI live inference backend, - and a React dashboard for run browsing and upload analysis.

Failure Cases

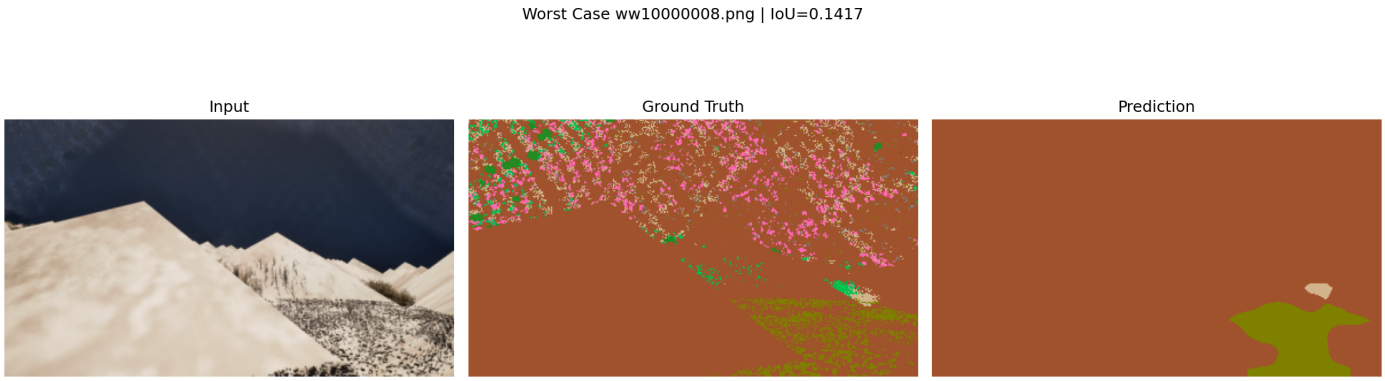


Failure Case 1

What happened:
The model struggled in a complex scene with mixed terrain and fine-grained regions.

Why it likely happened:
Rare and small classes receive limited supervision in the current CPU-friendly setup.

Potential fix:
Use class-balanced loss, stronger augmentation, and a richer decoder.



Failure Case 2

What happened:
Class boundaries were unstable in cluttered terrain.

Why it likely happened:
Low-resolution training and frozen features reduce sensitivity to small details.

Potential fix:
Use higher-resolution training on GPU and partial backbone fine-tuning.

---

5. Conclusion & Future Work

5.1 Deployment and Demonstration Layer

The project includes a usable demo layer in addition to offline evaluation, and the frontend now opens on the latest validation-leading run by default.

Backend: - FastAPI server for run discovery, asset serving, and upload inference

Frontend: - React dashboard for: - metric review, - split switching, - comparison view, - fullscreen analysis, - live upload inference, - and terrain suggestions beside the image

This improves the submission quality because judges can review both: - quantitative performance, - and qualitative model behavior on real scenes.

Conclusion

TerrainScope delivered a complete and reproducible segmentation kit with: - corrected label handling, - train and test pipelines, - validation and test evaluation, - full artifact export, - and a live dashboard for browsing runs and

uploading images.

The final system is strongest on: - Sky - Landscape - Dry Grass

The main performance gap remains in: - Logs - Rocks - Ground Clutter - Dry Bushes

Final headline metrics: - Best Validation Mean IoU: **42.88%** - Best Validation Mean Dice: **55.42%** - Best Validation Pixel Accuracy: **78.38%** - Best Test Mean IoU: **21.21%** - Best Test Mean Dice: **26.34%** - Best Test Pixel Accuracy: **61.66%** - Current Frontend Default Run: `probe_continue_high_iou`

Future Work

1. Train the continuation recipe uncapped on GPU to chase `0.50+` validation IoU.
2. Tune the already-implemented class-balanced loss and sampling to recover test generalization.
3. Fine-tune upper DINOv2 layers instead of freezing the full backbone.
4. Add targeted augmentation for clutter, occlusion, and obstacle-heavy scenes.
5. Compare against a stronger SegFormer run after the new optimizer/loss path.

---

6. Reproducibility and Submission Files

6.1 Key Output Artifacts

Important generated files for the latest report state: - `Offroad_Segmentation_Scripts/runs/probe_continue_high_iou/checkpoints/best_iou.pth` - `Offroad_Segmentation_Scripts/runs/probe_continue_high_iou/plots/training_metrics.png` - `Offroad_Segmentation_Scripts/runs/probe_continue_high_iou/evaluations/val_full_epoch8/evaluation_metrics.txt` - `Offroad_Segmentation_Scripts/runs/probe_continue_high_iou/evaluations/val_full_epoch8/per_class_iou.png` - `Offroad_Segmentation_Scripts/runs/probe_continue_high_iou/evaluations/val_full_epoch8/per_class_dice.png` - `Offroad_Segmentation_Scripts/runs/probe_continue_high_iou/evaluations/val_full_epoch8/confusion_matrix.png` - `Offroad_Segmentation_Scripts/runs/probe_continue_high_iou/evaluations/val_full_epoch8/comparisons/` - `Offroad_Segmentation_Scripts/runs/probe_continue_high_iou/evaluations/val_full_epoch8/worst_cases/` - `Offroad_Segmentation_Scripts/runs/quick_cpu_100x20/evaluations/Offroad_Segmentation_testImages/evaluation_metrics.txt`

Reproduction Commands

```
python -m pip install -r .\Offroad_Segmentation_Scripts\requirements.txt
python .\Offroad_Segmentation_Scripts\train.py --config .\Offroad_Segmentation_Scripts\configs\quick_cpu.jso
python .\Offroad_Segmentation_Scripts\train.py --config .\Offroad_Segmentation_Scripts\configs\quick_cpu.jso
python .\Offroad_Segmentation_Scripts\train.py --config .\Offroad_Segmentation_Scripts\configs\high_iou_cont
python .\Offroad_Segmentation_Scripts\test.py --config .\Offroad_Segmentation_Scripts\configs\high_iou_conti
python .\Offroad_Segmentation_Scripts\test.py --config .\Offroad_Segmentation_Scripts\configs\high_iou_conti
```

Submission Checklist

1. `Offroad_Segmentation_Scripts/train.py`
1. `Offroad_Segmentation_Scripts/test.py`
1. `Offroad_Segmentation_Scripts/configs/`
1. `Offroad_Segmentation_Scripts/offroad_segmentation/`
1. `Offroad_Segmentation_Scripts/runs/quick_cpu_100x20/`
1. `README.md`
1. `project_documentation.txt`
1. `report.md`
1. `final_report.md`

This report is the polished markdown version with percentage-based metrics and embedded charts for export.